



Università Degli Studi di Salerno
Ingegneria, Gestione ed Evoluzione del Software

ReFAIR

Toward a Context-Aware Recommender
for Fairness Requirements Engineering

Modification Report

Riferimento	Change Request Report Pre-modification System
Versione	1.1
Data	06/02/2025
Presentato da	PASTORE Carmine LAURINO Nicola

Sommario

- 1. Introduzione..... 3
- 2. Considerazioni preliminari..... 4
 - 2.1 Call graph 4
 - 2.2 Reachability matrix 5
- 3. Modifiche fatte..... 6
 - 3.1 CR2..... 7
 - Impact analysis 8
 - Implementazione..... 9
 - Reachability matrix aggiornata 10
 - 3.2 CR1 11
 - Impact analysis 12
 - Implementazione..... 13
 - 3.3 CR3..... 14
 - Impact analysis 15
 - Implementazione..... 17

1. Introduzione

Nel presente documento andiamo a documentare le modifiche effettuate al sistema Refair, le modifiche sono state avviate in seguito all'accettazione delle change request sottomesse.

Le modifiche effettuate mirano a migliorare la **scalabilità, la manutenibilità, le prestazioni e la chiarezza del codice**, con particolare attenzione alla transizione verso un'architettura più modulare e strutturata.

2. Considerazioni preliminari

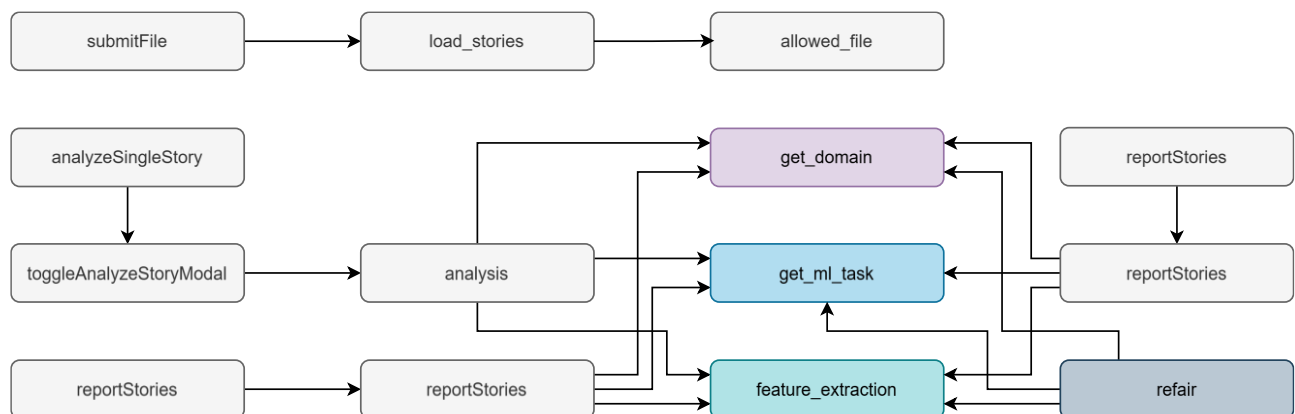
Per individuare tutte le componenti coinvolte nell'implementazione, e quindi impact analysis di ogni modifica proposta, sono state definite la matrice di raggiungibilità e il call graph. In particolare, la matrice di raggiungibilità è stata generata con lo scopo di definire il candidate impact set (CIS), ed è stato usato un approccio basato sulle distanze, con lo scopo di ridurre al minimo il numero di falsi positivi. Andiamo a prendere in considerazione tutte le componenti che interagiscono con le componenti in questione che si trovano a distanza ≤ 2 .

2.1 Call graph

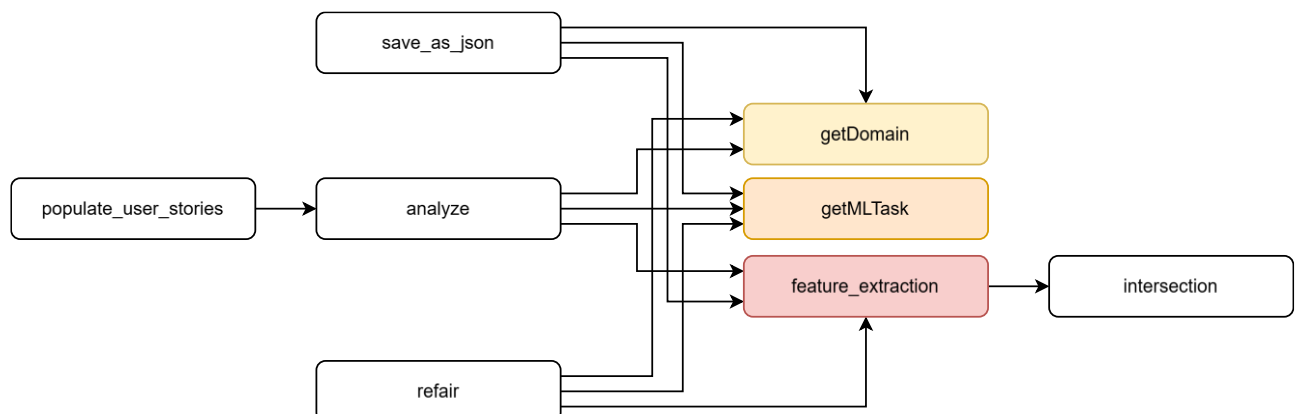
Il Call Graph rappresenta un grafo orientato che illustra le relazioni di chiamata tra le funzioni o i metodi all'interno del progetto Refair. Ogni nodo nel grafo corrisponde a una funzione, mentre un arco diretto collega due nodi quando una funzione richiama direttamente un'altra.

In quanto il la desktop app e il sistema server-client, sono due sistemi indipendenti sono stati realizzati due call graph.

Call graph server-client



Call graph desktop-app



2.2 Reachability matrix

Come fatto in precedenza per il call graph, anche in questo caso andremo a realizzare due matrici.

Componenti server-client	refair.py	app.py	refair.vue
<i>refair.py</i>		1	2
<i>app.py</i>	1		1
<i>refair.vue</i>	2	1	

Componenti desktop-app	index.py	domain_utils.py	user_story_canvas.py	custom_messagebox.py
<i>index.py</i>		1	1	1
<i>domain_utils.py</i>	1		2	2
<i>user_story_canvas.py</i>	1	2		1
<i>custom_messagebox.py</i>	1	2	1	

3. Modifiche fatte

La CR2 è stata eseguita prioritariamente in quanto prevedeva una revisione complessiva dell'architettura del sistema. Tale intervento era necessario per garantire la coerenza e la stabilità dell'infrastruttura prima dell'implementazione di ulteriori modifiche.

3.1 CR2

Nome		Ristrutturazione e riorganizzazione dell'architettura del sistema	
ID	CR2		
Tipologia CR	Manutentiva		
Descrizione	L'obiettivo principale è trasformare l'attuale sistema in una soluzione integrata, modulare e scalabile, riducendo al contempo la complessità di manutenzione e favorendo l'espandibilità futura. È prevista l'introduzione di un'architettura basata su microservizi per il back-end. Ogni servizio sarà responsabile di una funzione specifica e comunicherà attraverso API standardizzate. Questo approccio permetterà di separare le responsabilità, migliorare la scalabilità e facilitare gli aggiornamenti o le modifiche senza impattare l'intero sistema. La Desktop App verrà trasformata in un client connesso al sistema back-end, utilizzando le stesse API messe a disposizione per il client web. Questo cambiamento eliminerà la duplicazione di logiche e funzionalità, garantendo una maggiore coerenza tra le interfacce. Il front-end web, basato su Vue.js, verrà aggiornato per interfacciarsi con il nuovo sistema microservizi tramite un API Gateway. Questo consentirà di migliorare la modularità e la flessibilità per eventuali futuri sviluppi.		
Stato attuale	Attualmente, l'applicazione è organizzata in tre moduli principali: Versione Desktop: un'applicazione eseguibile direttamente sul computer dell'utente. Server Web: un servizio che espone un endpoint, consentendo di inviare input e ricevere output tramite chiamate HTTP. Client Web: un'interfaccia utente utilizzabile dal browser, che comunica con il server web per eseguire operazioni. L'applicazione, quindi, presenta due sistemi separati, ovvero l'applicazione desktop e la web-app. Non è prevista alcuna comunicazione tra l'app desktop e il sistema Client-Server, questo porta a una duplicazione delle logiche e delle funzionalità che non possono essere condivise tra i due sistemi. La duplicazione delle funzionalità comporta maggiori costi di sviluppo e manutenzione, aumentando il rischio di incoerenze tra i due sistemi. Il sistema Client-Server, invece, è costituito da un back-end monolitico, ciò comporta diverse criticità: - difficilmente scalabile, incrementi di carico o nuove funzionalità richiedono modifiche che possono impattare su tutto il sistema. - il client web dipende strettamente dalle API del server, riducendo la flessibilità.		
Benefici previsti	- Migliore scalabilità - Manutenibilità aumentata - Maggiore coerenza tra i due sistemi		
Proponenti	PASTORE Carmine LAURINO Nicola		
Data	24/12/2024	Versione	1.0

Impact analysis

La Change Request proposta prevede un completo rifacimento dell'architettura del sistema al fine di rispondere alle nuove esigenze funzionali e non funzionali, tra cui una maggiore scalabilità, manutenibilità e adattabilità alle evoluzioni future. Questo intervento implica una revisione profonda di tutte le componenti che riguardano il backend del sistema, poiché l'architettura rappresenta il fondamento strutturale su cui si basa l'intero sistema. Da un'analisi preliminare si deduce che la modifica coinvolgerà le seguenti componenti

SIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

CIS

In seguito ad un'analisi del call graph, e della Reachability matrix, è risalto che il CIS, nel caso in questione, risulta uguale al nostro SIS, in quanto, essendo un'operazione di reengineering a livello di design, un'operazione di re-design, è prevista una nuova architettura del sistema e questa modifica impatta sull'intero sistema.

CIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

AIS
refair.py
app.py
refair.vue
index.py
domain_utils.py

FPIS

Nessun componente.

DIS

Nessun componente.

Metriche

$$\text{Recall} = \frac{|CIS \cap AIS|}{|AIS|} = 1$$

$$\text{Precision} = \frac{|CIS \cap AIS|}{|CIS|} = 1$$

Implementazione

Le modifiche principali hanno riguardato la **scomposizione del backend monolitico**, l'**ottimizzazione della comunicazione con il client** e la **strutturazione di un'architettura a livelli**.

Per l'implementazione è stato deciso di migrare il backend verso un sistema orientato ai microservizi.

La prima fase del lavoro è stata identificare i vari servizi offerti dal server, in quanto il sistema attuale presentava tutte le funzionalità in un'unica codebase.

Successivamente sono stati sviluppati singolarmente ogni microservizio, è stato creato un package dedicato "*microservices*", e sono stati sviluppati i seguenti microservizi:

- Domain_prediction_service
- File_management_service
- Report_generation_service
- Task_prediction_service

I presenti servizi sono stati tutti estrapolati dal server esistente. Ogni servizio è stato sviluppato utilizzando **Flask**, con gestione delle dipendenze e della comunicazione tra servizi attraverso **REST API**. Per gestire le richieste ai vari microservizi, e semplificare il collegamento con il client è stato inserito un *api_gateway*, con il compito principale di instradare le richieste del client, verso il servizio dedicato. Così facendo i vari servizi possono essere eseguiti **in modo indipendente**, consentendo la scalabilità orizzontale. Una volta inserito l'*api_gateway* è stato modificato anche il lato client del sistema, le richieste del client ora vengono fatte all'*api_gateway* e non al server che era in funzione precedentemente.

Nella ristrutturazione dell'architettura, come ultimo punto, abbiamo modificato il funzionamento della **Desktop App**, che in precedenza gestiva **localmente** i modelli di machine learning, causando **ridondanza e difficoltà di aggiornamento**. Con la nuova architettura, la Desktop App ora comunica direttamente con i microservizi **tramite l'API Gateway**, eliminando la necessità di mantenere i modelli localmente, garantendo un accesso centralizzato e garantendo coerenza tra i due sistemi.

Reachability matrix aggiornata

A seguito della modifica dell'architettura, si è reso necessario ridisegnare la matrice di raggiungibilità, in quanto l'introduzione di nuovi file per la gestione del sistema ha modificato le relazioni e le dipendenze tra i componenti.

Componenti	refair.py	app.py	refair.vue	Domain_prediction_service.py	File_management_service.py	Report_generation_service.py	Task_prediction_service.py	Api_gateway.py	index.py	domain_utils.py	user_story_canvas.py	custom_messagebox.py
refair.py		1										
app.py	1											
refair.vue				2	2	2	2	1				
Domain_prediction_service.py			2			1		1	1			
File_management_service.py			2					1	1			
Report_generation_service.py			2	1			1	1	1			
Task_prediction_service.py			2			1		1	1			
Api_gateway.py			1	1	1	1	1		1			
index.py				2	2	2	2	1				
domain_utils.py												
user_story_canvas.py									1	2		1
custom_messagebox.py									1	2	1	

3.2 CR1

Nome Implementazione di HITL (Human-in-the-loop) per il modulo di Machine Learning			
ID	CR1		
Tipologia CR	Evolutiva		
Descrizione	La presente change request mira ad introdurre un meccanismo di HITL* nel sistema esistente con lo scopo di migliorare l'accuratezza dei modelli predittivi tramite il coinvolgimento attivo degli utenti, che, una volta ottenuto il risultato, dovranno fornire un feedback, sotto forma di voto, sull'accuratezza del dato.		
Stato attuale	Attualmente, il sistema utilizza modelli di machine learning pre-addestrati e produce risultati basati esclusivamente su un dataset fornito in fase di addestramento. Tuttavia, se il dataset di addestramento non viene aggiornato periodicamente con nuovi esempi ed effettuato il retraining del modello, c'è il rischio che il modulo produca output meno accurati nel tempo, specialmente in presenza di dati o scenari non previsti inizialmente. Il sistema, inoltre, non prevede un meccanismo per la raccolta dei feedback degli utenti.		
Benefici previsti	Migliore accuratezza da parte del sistema grazie all'addestramento continuo da parte dei modelli di machine learning. Maggiore capacità del sistema di adattarsi a nuovi scenari o variazioni nei dati.		
Proponenti	PASTORE Carmine LAURINO Nicola		
Data	24/12/2024	Versione	1.0

Impact analysis

SIS
Refair.vue
User_story_canvas.py
Api_gateway.py

CIS
Refair.vue
User_story_canvas.py
Api_gateway.py
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Index.py

AIS
Refair.vue
User_story_canvas.py
Api_gateway.py

FPIS
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Index.py

DIS

Nessun componente.

Metriche

$$\text{Recall} = \frac{|CIS \cap AIS|}{|AIS|} = 1$$

$$\text{Precision} = \frac{|CIS \cap AIS|}{|CIS|} = 3/8 = 0.375$$

Implementazione

Per l'implementazione della presente CR, è stato necessario aggiungere un altro servizio, ovvero il feedback. Il feedback viene rilasciato dagli utenti in seguito ad un'analisi di una determinata US, una volta analizzata, gli utenti possono scegliere se lasciare una valutazione, con un voto da 1 a 5, alla predizione del modello, oppure no. Una volta ottenuto il feedback, sulla predizione il modello lo salva per eseguire in secondo momento una fase di retraining del modello.

I feedback vengono salvati in due file json differenti, uno per i feedback del task prediction e uno per i feedback per il domain prediction.

Come prima fase sono state apportate modifiche al frontend del client e della desktopApp, per implementare il sistema di valutazione attraverso una modal.

Successivamente è stato implementato il sistema di retraining, il sistema di retraining viene attivato al raggiungimento di una soglia prestabilita di feedback, questo perché allenarlo con pochi feedback potrebbe portare ad un effetto indesiderato, ovvero rovinare il modello invece di migliorarlo. Una volta che viene raggiunta la soglia limite, il sistema avvia il retraining, combinando il dataset originale, e i feedback ricevuti dagli utenti, questo viene fatto per evitare che il modello dimentichi le informazioni importanti del dataset di partenza. Ora i feedback ricevuti saranno nuovi dati di addestramento del modello e ciascun dato viene ponderato (pesato), in base al valore di feedback ricevuto, se il valore di feedback è basso (1-2) allora significa che il modello ha sbagliato predizione e quindi questi dati avranno peso maggiore in fase di retraining, se invece hanno avuto un voto alto, significa che il modello sta lavorando bene e questi dati avranno un peso minore in fase di retraining.

Una volta terminato l'addestramento il modello, calcola la metrica F1-score, se questa è **migliore** del modello precedente, ovvero prima dell'addestramento, lo sostituisce e svuota la lista dei feedback, altrimenti se il nuovo modello ha un punteggio **inferiore**, il retraining viene annullato e il modello precedente rimane attivo.

Questo processo viene fatto allo stesso modo sia per il *domain prediction* e sia per il *task prediction*.

Per implementare il sistema di retraining, è stato aggiunto un nuovo servizio, chiamato *feedback_service*, con due endpoint messi a disposizione del client ovvero */feedback/domain* e */feedback/task*.

È stato creato un nuovo package, *feedback_utils*, che contiene tutto ciò che serve per eseguire il retraining e gestire i feedback dell'utente.

Come ultimo passaggio è stato effettuato il collegamento tra backend e frontend.

3.3 CR3

Nome	Refactoring del codice		
ID	CR3		
Tipologia CR	Manutentiva		
Descrizione	L'obiettivo di questa attività è semplificare il codice sorgente e documentare funzioni e classi con lo scopo di rendere il codice più leggibile, mantenibile e conforme alle best practice dello sviluppo software. Applicando i principi SOLID, sarà possibile ottenere un codice più semplice, pulito e facilmente estendibile per futuri sviluppi. Inoltre, la documentazione del codice permetterà di facilitare il primo approccio al progetto, offrendo una guida chiara e dettagliata sul funzionamento delle classi e dei metodi. È prevista una “pulizia” del codice. Questo renderà ReFAIR un'applicazione più mantenibile e pronta per l'implementazione di nuove funzionalità. Inoltre, l'attività comprende azioni volte alla rimozione di classi e metodi non utilizzati, che comprendono anche azioni di pulizia e semplificazione del codice. In questo modo, si ottiene un codice più semplice e mantenibile.		
Stato attuale	• Il codice sorgente attuale non è documentato, rendendo difficile capire il funzionamento di classi e metodi. Non sono presenti informazioni sui parametri richiesti o sugli output forniti. • La mancanza di documentazione complica il primo approccio al progetto, rendendo poco intuitivo comprendere il funzionamento complessivo dell'applicazione. • Non è specificato l'elenco delle librerie richieste per l'esecuzione né la versione di Python necessaria per il corretto funzionamento del progetto. • Pochi commenti rendono il codice difficile da interpretare e da leggere. • Sono presenti classi e metodi non utilizzati, inizialmente previsti unicamente per testare le funzionalità implementate.		
Benefici previsti	Migliore manutenibilità e leggibilità del codice sorgente.		
Proponenti	PASTORE Carmine LAURINO Nicola		
Data	24/12/2024	Versione	1.0

Impact analysis

L'operazione in corso consiste in un'attività di refactoring del codice, con l'obiettivo di migliorare la qualità e la manutenibilità del software senza alterarne il comportamento funzionale. Il refactoring include una revisione approfondita della documentazione del codice, assicurando chiarezza e coerenza nelle descrizioni delle funzionalità e delle interazioni tra i moduli. Inoltre, verranno applicati i principi SOLID, con particolare attenzione alla separazione delle responsabilità, all'uso di interfacce e all'indipendenza dei moduli, al fine di rendere il sistema più modulare ed estensibile.

SIS

L'impatto di questa operazione interesserà tutte le componenti del sistema, poiché la ristrutturazione del codice può influenzare dipendenze, interfacce e logiche di business. Quindi il nostro start impact set sarà formato da tutte le componenti del sistema.

SIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

CIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

AIS
refair.py
app.py
refair.vue
Domain_prediction_service.py
File_management_service.py
Report_generation_service.py
Task_prediction_service.py
Api_gateway.py
index.py
domain_utils.py
user_story_canvas.py
custom_messagebox.py

FPIS

Nessun componente.

DIS

Nessun componente.

Metriche

$$\text{Recall} = \frac{|CIS \cap AIS|}{|AIS|} = 1$$

$$\text{Precision} = \frac{|CIS \cap AIS|}{|CIS|} = 1$$

Implementazione

Il codice del sistema presentava un forte accoppiamento tra le varie componenti, presentava criticità in termini di leggibilità e ciò rendeva complessa la sua comprensione.

L'applicazione dei principi ha reso necessario una ristrutturazione dei microservizi realizzati. Innanzitutto, si è cominciato analizzando nel dettaglio i microservizi generati e se questi rispettavano o meno i principi. Di seguito, un resoconto dell'analisi effettuata.

	Domain Prediction Service	Task Prediction service	File Management service	Report Generation service	Feedback service
<i>SRP</i>	✗	✗	✗	✗	✗
<i>OCP</i>	✓	✓	✓	✓	✓
<i>LSP</i>	✓	✓	✓	✓	✓
<i>ISP</i>	Non applicabile	Non applicabile	Non applicabile	Non applicabile	Non applicabile
<i>DIP</i>	✗	✗	✓	✗	✗

Sigla	Descrizione
<i>SRP</i>	Single responsibility principle Ogni classe dovrebbe avere una ed una sola responsabilità.
<i>OCP</i>	Open/closed principle Un'entità software dovrebbe essere aperta alle estensioni, ma chiusa alle modifiche.
<i>LSP</i>	Liskov substitution principle Gli oggetti dovrebbero poter essere sostituiti con dei loro sottotipi, senza alterare il comportamento di chi li utilizza.
<i>ISP</i>	Interface segregation principle Sarebbero preferibili più interfacce specifiche.
<i>DIP</i>	Dependency inversion principle Una classe dovrebbe dipendere dalle astrazioni, non da classi concrete.

Applicazione principi SOLID per Domain Prediction Service

Le funzionalità sono state estrapolate e inserite in classi specifiche, così da rispettare il principio della singola responsabilità. Inoltre, per rispettare il principio di inversione delle dipendenze, i classificatori utilizzati sono stati astratti e generalizzati una classe specifica così da ottenere la possibilità di poter cambiare modello in qualsiasi momento senza dover modificare la classe ma aggiornando semplicemente un file di configurazione.

Applicazione principi SOLID per Task Prediction service

Come fatto per il microservizio precedente, anche il servizio di previsione dei task sono state estratte le funzionalità principali ed inserite in classi specifiche. Inoltre, la scelta del modello di classificazione viene gestita da una classe specifica.

Applicazione principi SOLID per File Management Service

La logica del microservizio è stata divisa in sottoclassi così da ottenere delle classi specifiche per il caricamento e il controllo delle estensioni.

Applicazione principi SOLID per Report Generation Service

Il microservizio effettua l'invocazione dei servizi di classificazione di dominio e task direttamente nella sua logica di business, realizzando di fatto una forte dipendenza con i microservizi citati. La classe è stata allora divisa così da realizzare singole classi con specifiche responsabilità e le invocazioni dei microservizi di classificazione sono stati astratti e generalizzati, così un cambio dei servizi di classificazione non incide direttamente sul funzionamento del microservizio in oggetto.

Applicazione principi SOLID per Feedback Service

La logica del servizio è stata divisa in classi così da ottenere classi con responsabilità specifiche. Le dipendenze con le librerie utilizzate per la gestione dei dati sono state spostate nelle singole classi semplificando di fatto le dipendenze dirette nel servizio.

Di seguito, il resoconto dell'applicazione dei principi SOLID dopo aver aggiornato il progetto con le modifiche sopra descritte.

	Domain Prediction Service	Task Prediction service	File Management service	Report Generation service	Feedback service
<i>SRP</i>	✓	✓	✓	✓	✓
<i>OCP</i>	✓	✓	✓	✓	✓
<i>LSP</i>	✓	✓	✓	✓	✓
<i>ISP</i>	<i>Non applicabile</i>	<i>Non applicabile</i>	<i>Non applicabile</i>	<i>Non applicabile</i>	<i>Non applicabile</i>
<i>DIP</i>	✓	✓	✓	✓	✓

Per favorire la leggibilità e la manutenibilità del codice e delle varie sezioni che compongono l'applicazione ReFAIR, ogni singolo metodo è stato accuratamente commentato e spiegato. Per favorire la facilità di utilizzo, gli endpoint esposti dalla api gateway sono state accompagnati da commenti specifici che mostrano come utilizzare il servizio: il json request da inviare e il json response ottenuto. L'organizzazione della struttura del progetto è stata rivista, ponendo maggior attenzione a una netta divisione tra client e server, eliminando codice e file non più utilizzati. In particolare, la nuova struttura proposta è così organizzata:

- **Server**

Il codice responsabile della logica di funzionamento dell'applicazione, ovvero il core di ReFAIR, è stata organizzata sotto il pacchetto *server*. L'organizzazione interna è stata a sua volta raffinata, organizzando il contenuto in:

- *Api gateway*

Contiene il codice Python che si occupa di fornire gli end point utilizzati per l'invocazione dei metodi principali di ReFAIR.

- *Env*

Per semplicità di utilizzo, sono stati definiti gli ambienti virtuali per ciascun microservizio e per l'api gateway.

- *Microservices*

Contiene i microservizi realizzati per l'esecuzione della logica di business di ReFAIR. Inoltre, come precedentemente descritto, le singole responsabilità di ciascun microservizio sono state estrapolate e organizzate in classi specifiche, organizzate nella sottodirectory *services*.

- *Utils*

La cartella viene utilizzata per lo store di tutti quei file utilizzati per il funzionamento di ReFAIR, quali i dataset di addestramento, i modelli, i classificatori e i feedback.

- **Client**

Le componenti che si occupano di fornire un'interfaccia grafica per l'utilizzo dei servizi di ReFAIR sono stati organizzati in questa cartella.

- *Desktop app*

Contiene la logica di funzionamento dell'applicazione desktop realizzata per ReFAIR.

- *Web app*

Contiene i file necessari per l'esecuzione dell'applicazione client web.

- **System Test**

La cartella mantiene tutte le classi realizzate per i testing di sistema.

- **Image e Documentation**

Tutti la documentazione realizzata per descrivere il progetto ReFAIR sono organizzate nelle cartelle *images* e *IGES Documentation*.

Questa nuova organizzazione di ReFAIR consente di avere una netta separazione tra server, client e documentazione, rendendo di fatto più leggibile e mantenibile l'intero ecosistema di ReFAIR.

